Chapter 12

Software Tools and Emerging Technologies for Vehicle Routing and Transportation

Vehicle Routing: Problems, Methods, and Applications, 2nd ed. (2014)

Based on: Ola Braysøy & Geir Hasle

Slides prepared for self-contained study

2026

Chapter Overview

This chapter surveys the **software landscape** for Vehicle Routing Problems (VRPs): what commercial and open-source tools exist, what algorithmic engines they use, how they handle data and user interaction, and where the technology is heading.

Outline

12.1 Introduction I

Vehicle routing is a central problem for a large number of public and private corporations, including logistics companies, parcel delivery services, transit agencies, and solid-waste collection firms. Solving real VRP instances well can cut transport costs by **5–30%**, reduce fuel consumption and CO_2 emissions, and improve customer service levels.

Why software matters more than algorithms alone

The academic literature has produced dozens of powerful exact methods, heuristics, and metaheuristics for VRP. However, a mathematical algorithm by itself is not directly useful to a practitioner: it must be embedded in a *software product* that:

- reads real-world problem data (addresses, time windows, vehicle specs),
- integrates with mapping and GIS (Geographic Information System) layers,
- enforces dozens of practical business constraints,
- presents routes to planners and drivers in an understandable form, and
- can be maintained and extended over time.

12.1 Introduction II

The gap between research and practice

Research algorithms are often developed and tested on *benchmark instances* (e.g., Solomon's 100-customer VRPTW instances) under idealised assumptions. Real-world deployments involve:

- thousands of customers per day,
- dozens of interacting constraints (driver breaks, hazmat, vehicle compatibility, customer preferences),
- uncertain travel times (traffic), dynamic new orders, and
- the need for human planners to understand, trust, and override the software.

Bridging this gap is the central challenge of *VRP software technology*.

This chapter focuses on **applied** aspects: it does not present new algorithms, but instead analyses how the best existing algorithms are integrated into commercial products and what practical requirements drive software design decisions.

12.1 Introduction III

Scope of the survey

The authors conducted a questionnaire-based survey of VRP software vendors, receiving responses from nine companies operating worldwide (Table 12.1, discussed in Section 12.7). The survey covered:

1. system architecture and user interfaces,
2. input/output capabilities and data integration,
3. supported VRP model variants,
4. embedded algorithms and solvers,
5. implementation platform, performance benchmarks, and pricing, and
6. adoption of emerging technologies.

The chapter also reviews the open-source and academic software landscape and discusses future directions.

12.1 Introduction IV

Key insight: industrial VRP is complex

Vendors report that the typical industrial VRP involves far more than distance minimisation: the problem includes driver working-hour regulations, vehicle compatibility rules, geographic zoning, multi-day planning horizons, and real-time order changes. A routing algorithm that cannot handle these aspects cannot be deployed in practice, regardless of its theoretical performance.

Outline

12.2 Basic Functionalities of VRP Routing Software I

Every VRP software product, whether commercial or open-source, must provide a core set of functionalities. Understanding these functionalities helps us evaluate and compare tools.

12.2 Basic Functionalities of VRP Routing Software II

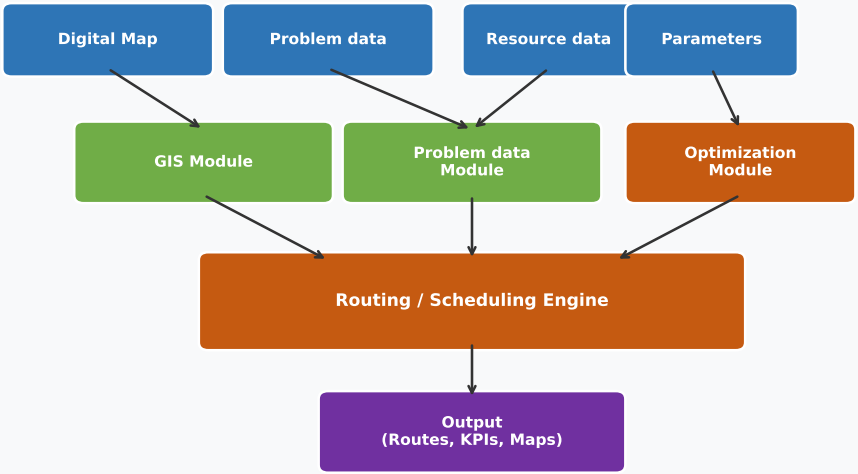
Core components of a VRP routing system

A routing system typically comprises four main modules:

1. **GIS (Geographic Information System) Module** — integrates map data, computes travel times and distances between locations, geocodes addresses (converts street addresses to latitude/longitude coordinates).
2. **Problem Data Module** — stores and manages customer data (locations, demands, time windows, priorities), vehicle data (capacity, type, availability), and depot information.
3. **Optimization Module** — the algorithmic engine that computes routes; may include construction heuristics, local search, metaheuristics, or exact solvers.
4. **Output/Reporting Module** — displays routes on maps, generates driver turn-by-turn directions, exports data to ERP (Enterprise Resource Planning) and TMS (Transport Management System) systems, and computes KPIs (Key Performance Indicators) such as total distance, cost, and service level.

12.2 Basic Functionalities of VRP Routing Software III

Figure 12.1 — Architecture of VRP Routing Software



Outline

12.3 Input and Output I

A VRP system must be able to read data from the user's existing IT infrastructure and return results in formats that integrate with downstream systems. Poor I/O capabilities are often a bigger barrier to adoption than algorithmic limitations.

Typical input data formats

- **Spreadsheets** (CSV, Excel) — the most common entry point for smaller operations; customer lists with addresses, demands, and time windows.
- **ERP/TMS system integration** — via APIs, ODBC/JDBC database connections, or XML/JSON feeds from systems such as SAP, Oracle, or Microsoft Dynamics.
- **EDI (Electronic Data Interchange)** — standardised electronic business documents (orders, invoices) in formats such as EDIFACT or ANSI X12.
- **GPS track logs** — historical vehicle positions in formats such as NMEA, GPX, or vendor-specific binary formats; used for driver performance analysis and model calibration.

Map and GIS data

A routing system needs a *road network* to compute realistic travel times and distances. Sources include:

- **Commercial digital maps** — HERE (formerly Navteq), TomTom, Google Maps Platform; typically licensed annually. Provide turn restrictions, road speeds, truck-height restrictions, etc.
- **OpenStreetMap (OSM)** — free, crowd-sourced global map; quality varies by region; sufficient for many applications.
- **Proprietary road-speed databases** — some vendors combine map data with historical GPS probe data to provide time-of-day-dependent travel times (e.g., faster at 2 am than at 8 am rush hour).

12.3 Input and Output III

Geocoding

Geocoding is the process of converting a street address such as "75 Port-Royal Street East, Montreal" into geographic coordinates (latitude, longitude). Accurate geocoding is essential: a geocoding error of even 100 metres can cause the system to assign a customer to the wrong road segment, leading to incorrect travel times and infeasible routes. Most commercial systems include or interface with a geocoding service.

formed on instances with size and complexity that are comparable to industrial requirements. There is a tendency towards more cooperation between researchers in industry and academia. Furthermore, there is no strict segregation between the developers of industrial VRP tools and researchers in academia; some have positions in both camps, and some companies are tightly connected to groups in academia. However, there is still a considerable way to go to coordinate the efforts in these communities.

The trend towards more industrial relevance has led to a new category of VRP research. The term "Rich VRP" (Hasle and Kloster [33] and Drexl [13]) has been coined to describe a more holistic approach where one defines a more generic model that adequately captures all of the most important aspects of a selected segment of industrial applications. A main goal is to develop solution algorithms that produce high quality solutions in reasonable time for any instance of the generic model. Often, one tries to achieve this "instance robustness" with a uniform algorithmic approach.

VRPs are, in essence, highly complex optimization problems. Because of this complexity, software for supporting human planners and decision makers has been widely used for years. Computerized vehicle routing helps businesses to improve the utilization of their transportation resources. It can help to reduce journey times, vehicle mileage, and costs, but also increase revenues and improve customer service. This is achieved by rapidly processing the information concerning customer locations and quantities and types of goods to be transported and matching these to available vehicle capacity in order to make best use of all resources. In addition, users obtain substantial customer service benefits

Figure: Screenshot of a typical VRP routing software interface (Figure 12.2 from the book). The map view shows planned routes superimposed on a road network, with coloured route segments and depot/customer markers.

Outline

12.4 Model Properties — Supported VRP Variants I

A VRP software product is only useful if it can model the constraints that arise in the user's actual operations. This section surveys the main constraint types supported by commercial tools.

12.4.1 Basic setup: order types

The most fundamental choice is the **order type**:

- **Pickup-only** (e.g., collecting goods from suppliers).
- **Delivery-only** (e.g., delivering parcels from depot to customers) — the most common case.
- **Mixed pickup and delivery** — vehicles collect goods from some locations and deliver to others; the two types may be interleaved on the same route.
  - *Simultaneous pickup and delivery (SPDP)*: at each customer the driver both delivers and picks up.
  - *Paired pickup-and-delivery (PDP)*: each pickup location is paired with a specific delivery location (e.g., courier, dial-a-ride).

12.4 Model Properties — Supported VRP Variants II

Route start and end flexibility

In the classical VRP, all vehicles start and end at the same depot. Real operations often require:

- **Multiple depots** — several warehouses or terminals.
- **Open routes** — vehicles end at their last customer (driver goes home) rather than returning to the depot.
- **Multiple time horizons** — routes spanning several days, e.g., long-haul trucking.

Planning Horizon

One frequently encountered complication is that planning must be done over more than a single day. For example, a wholesale distributor may visit some customers only on Mondays and Wednesdays, others on Tuesdays and Thursdays. This leads to **period VRP** variants, supported by several commercial systems.

12.4.2 Uncertainty and Dynamics

Uncertainty. Real customer demands are often not known exactly in advance. A system must handle:

- *Stochastic demands* — the actual demand at customer i is a random variable $\tilde{d}_i$ (d-tilde sub i) with known probability distribution; the route must be feasible with high probability.
- *Dynamic orders* — new customer orders arrive during the planning day and must be inserted into existing routes.

Delivery Day. Some customers require delivery on a specific day or on one of several allowed days. For example, a dairy product may need to be delivered within 24 hours of order.

Priorities. In practice, customers differ in importance. Priority constraints allow the planner to specify that high-priority customers must be served first or within a tighter time window. Some tools support *soft* priorities that can be violated at a penalty cost.

Same-Tour Constraints and Time Limits

Same-Tour Constraints. Some business rules require that:

- certain customers always travel on the *same* route (same vehicle), e.g., hazmat customers grouped with a certified driver; or
- certain customers must *never* be on the same route (competing customers who must not see each other's orders).

Time Limits. Maximum route duration (total time on road) and maximum shift length for drivers are standard constraints. Some systems also support:

- mandatory rest breaks (e.g., EU regulations require a 45-minute break after 4.5 hours of driving),
- overtime rules and associated cost penalties.

Time Windows (VRPTW)

Time windows are among the most commonly used and most impactful constraints in real VRP applications.

A **time window** $[a_i, b_i]$ for customer i specifies that service must begin no earlier than time a_i (a sub i, the *opening time*) and no later than time b_i (b sub i, the *closing time*). Here a and b are typically given in minutes or hours from the start of the planning horizon (e.g., 08:00 = 0, 09:00 = 60).

- *Hard time windows*: arriving after b_i is infeasible; the customer cannot be served.
- *Soft time windows*: arriving after b_i incurs a penalty cost proportional to the lateness $\max(0, t_i - b_i)$, where t_i (t sub i) is the actual arrival time.

Most commercial tools support both hard and soft time windows.

12.4 Model Properties — Supported VRP Variants VI

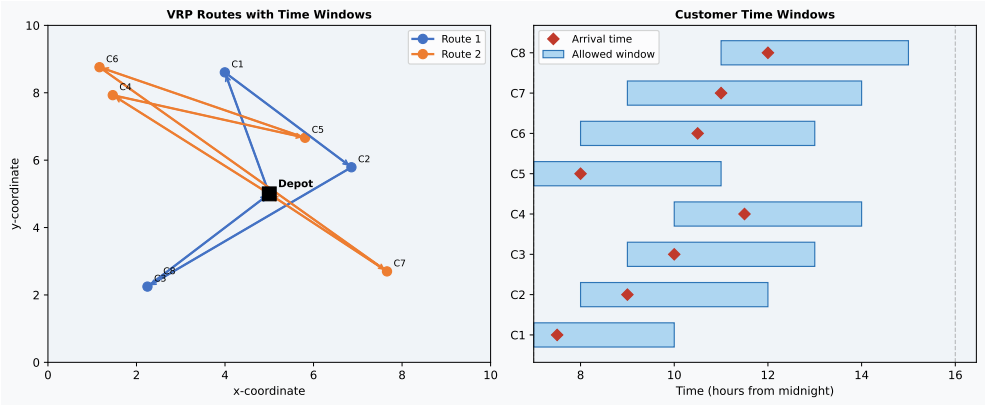


Figure: Left: two optimised routes for 8 customers. Right: each customer's allowed delivery window (blue bar) and the planned arrival time (red diamond). All arrivals fall within the windows in this example.

12.4 Model Properties — Supported VRP Variants VII

Capacity Constraints

The most basic VRP constraint is vehicle **capacity**: the total load carried on a route must not exceed the vehicle's maximum capacity Q (Q, a positive number representing weight, volume, or number of items). Many real applications involve *multiple capacity types* simultaneously. For example, a food-distribution vehicle may be limited by:

- *weight* (e.g., maximum 5 tonnes),
- *volume* (e.g., maximum 20 cubic metres), and
- *number of pallets* (e.g., maximum 33 Euro-pallets).

All three constraints must hold simultaneously.
Most commercial VRP systems support multiple capacity dimensions. Open-source systems often support only a single dimension.

12.4 Model Properties — Supported VRP Variants VIII

Temporal Availability

Customers, drivers, and vehicles may each have *availability windows*: time intervals during which they can participate in a route. A vehicle may be available only from 06:00 to 22:00; a driver may work only a morning or afternoon shift. These constraints interact with time windows to make the feasibility check non-trivial.

12.4 Model Properties — Supported VRP Variants IX

Compatibility Constraints

In many industries, not every vehicle can serve every customer:

- A *refrigerated truck* is required for temperature-sensitive goods.
- A *tail-lift vehicle* is needed for customers without loading docks.
- A *certified hazmat vehicle and driver* are legally required for dangerous goods.
- *Vehicle size restrictions* apply on narrow roads or in pedestrian zones.

These are modelled as **compatibility matrices**: a binary table specifying which (vehicle, customer) pairs are compatible.

Driving Regulations

European and North American regulations impose mandatory rest periods. The EU drivers' hours rules (Regulation (EC) No 561/2006) state:

- After 4.5 hours of continuous driving, a 45-minute break is required.
- The daily driving limit is 9 hours (extendable to 10 hours twice a week).
- 11 hours of daily rest are mandatory.

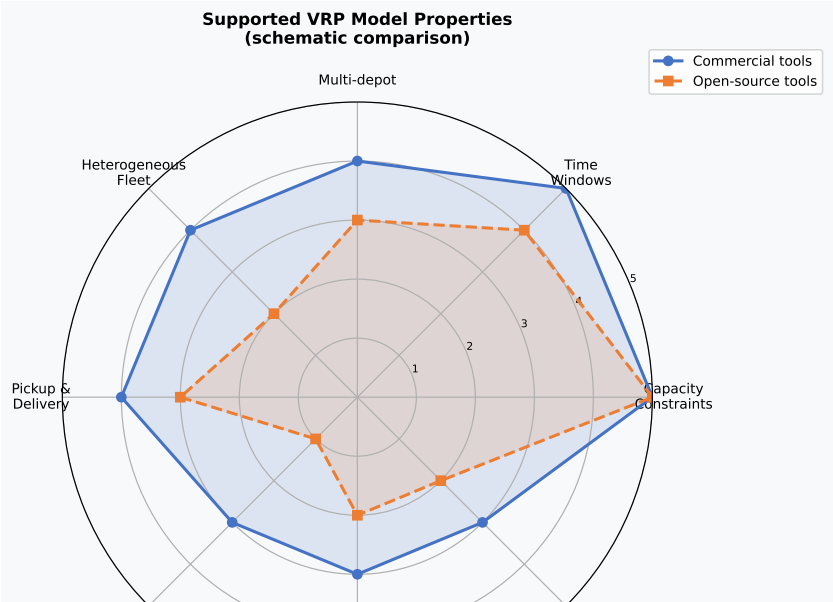
Commercial VRP software must model these rules and insert required breaks at feasible locations on each route. Only a few systems handle this fully.

Zoning

Zoning refers to geographic or administrative groupings that constrain how customers are assigned to vehicles or routes:

- *Predefined service zones* — a delivery region is divided into zones, each assigned to a specific vehicle or driver. Useful for driver familiarity and customer relationship management.
- *Zone balancing* — the system aims to keep zones roughly equal in workload across multiple planning periods.
- *Zone optimisation* — the system redesigns zone boundaries along with routes, a more powerful but computationally harder approach.

12.4 Model Properties — Supported VRP Variants XII



Outline

12.5 Algorithms in Commercial VRP Software I

Information on the algorithms used inside commercial VRP tools is usually not publicly disclosed, as it is a proprietary asset. However, the survey results and published research allow some conclusions. The dominant methods in commercial systems are **metaheuristics**, with some exact methods for small sub-problems.

12.5 Algorithms in Commercial VRP Software II

Most common algorithmic strategies

1. **Construction heuristics** — build an initial feasible solution quickly (seconds or less), even if suboptimal. Used as a starting point for improvement. Examples: Clarke-Wright Savings, Nearest Neighbour insertion, Sweep algorithm.
2. **Local search / improvement heuristics** — iteratively improve the current solution by making small changes (moves). A move is accepted if it reduces the objective (total distance/cost). Terminates at a local minimum. Examples: 2-opt, 3-opt, Or-opt (relocate one or two customers).
3. **Metaheuristics** — sophisticated search strategies that can escape local minima by sometimes accepting worse solutions or by exploring multiple solutions simultaneously. Examples: Tabu Search, Simulated Annealing, Genetic Algorithms, ALNS (Adaptive Large Neighbourhood Search).
4. **Exact methods** — solve a mathematical programming formulation to provable optimality; practical only for small instances (up to $\sim$100 customers) or small sub-problems. Examples: Branch-and-Bound, Branch-and-Cut, Column Generation.

12.5 Algorithms in Commercial VRP Software III

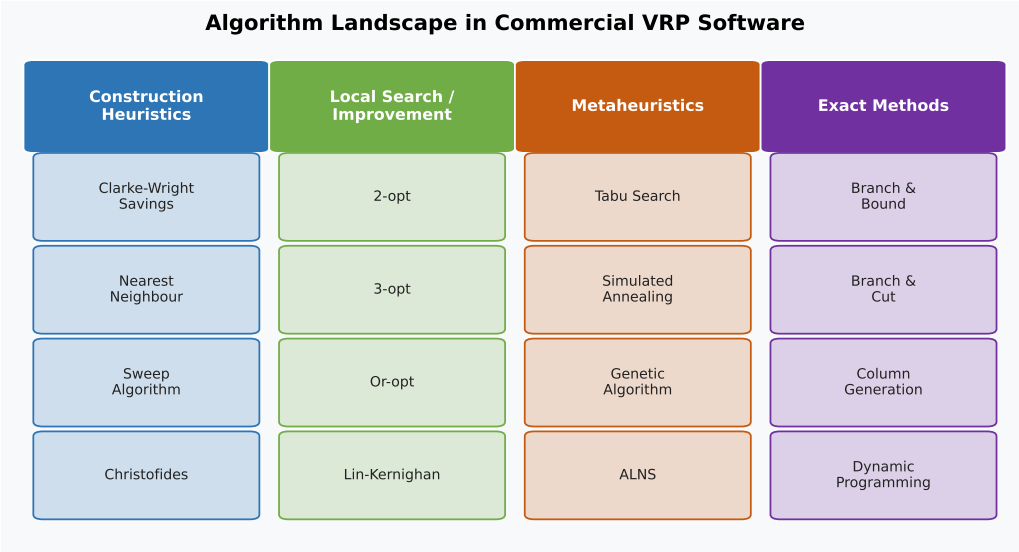


Figure: The algorithm landscape in commercial VRP software. Construction heuristics provide a fast starting point,

Clarke-Wright Savings: Python Implementation

Listing 1: Clarke-Wright Savings Algorithm in Python

```python
import numpy as np
from itertools import combinations

def clarke_wright(depot, customers, demands, capacity):
    """
    depot    : (x, y) coordinates of the depot
    customers: dict {id: (x, y)}
    demands  : dict {id: demand}
    capacity : vehicle capacity Q
    Returns  : list of routes, each a list of customer ids
    """
    ids = list(customers.keys())
    def dist(a, b):
        return np.linalg.norm(np.array(a) - np.array(b))

    # Step 1: compute savings s(i,j) = d(depot,i) + d(depot,j) - d(i,j)
    savings = []
    for i, j in combinations(ids, 2):
        s = (dist(depot, customers[i]) + dist(depot, customers[j])
             - dist(customers[i], customers[j]))
        savings.append((s, i, j))
    savings.sort(reverse=True)  # largest saving first

    # Step 2: initialise individual routes
    routes     = {i: [i] for i in ids}      # route key -> stop list
```

ALNS Skeleton for CVRP: Python

Listing 2: Adaptive Large Neighbourhood Search (ALNS) skeleton

```python
import random, math, copy

def alns_cvrp(depot, customers, demands, capacity,
              n_iter=5000, q=5, T0=100.0, alpha=0.997):
    """
    ALNS for Capacitated VRP.
    alpha (alpha) : cooling rate -- temperature T decreases as T *= alpha each step.
    T0            : initial temperature (controls how often worse solutions are accepted).
    q             : number of customers removed in each destroy step.
    """
    def route_cost(routes):
        total = 0
        for r in routes:
            prev = depot
            for c in r:
                total += dist(prev, c); prev = c
            total += dist(prev, depot)
        return total

    def destroy(routes, q):
        flat = [c for r in routes for c in r]
        to_remove = set(random.sample(flat, min(q, len(flat))))
        new_routes = [[c for c in r if c not in to_remove] for r in routes]
        return [r for r in new_routes if r], list(to_remove)
```

2-opt Local Search: Python Implementation

Listing 3: 2-opt improvement for a single VRP route

```python
import numpy as np

def two_opt(route, dist_matrix):
    """
    Improve a single route using 2-opt.
    route       : list of customer indices (depot NOT included)
    dist_matrix : 2D array where dist_matrix[i][j] is travel cost i->j

    A 2-opt move removes two edges (a->b) and (c->d) and reconnects
    as (a->c) and (b->d), reversing the segment between b and c.
    This eliminates crossing edges, always reducing total distance.
    """
    def route_cost(r):
        full = [0] + r + [0]     # 0 = depot
        return sum(dist_matrix[full[k]][full[k+1]]
                   for k in range(len(full)-1))

    improved = True
    best = route[:]
    while improved:
        improved = False
        for i in range(len(best) - 1):
            for j in range(i + 2, len(best)):
                # Reverse segment best[i+1 : j+1]
                candidate = (best[:i+1]
```

Outline

12.6 Implementation, Performance, and Price I

This section summarises practical information about commercial VRP software products: what platform they run on, how they perform on standard benchmarks, and what they cost.

Implementation platform

According to the survey:

- **Almost all** commercial VRP systems are implemented in C++ or Java, occasionally in C#/.NET. Python is rarely used for the core solver (too slow) but common for scripting and integration layers.
- **Windows** is the dominant target platform; most systems run natively on Windows and offer a Windows-based GUI.
- A growing number offer a **web-based** or **cloud-hosted** interface, accessible from any browser — important for SaaS (Software as a Service) deployments.
- Linux support is available in some systems for server-side batch processing.

Multi-threading and performance

Modern VRP solvers exploit multi-core processors. The most advanced systems run **parallel search threads** that:

- explore different regions of the solution space simultaneously,
- exchange the best found solutions between threads periodically, and
- adaptively redirect search effort toward promising regions.

GPU (Graphics Processing Unit) computing is not yet common in commercial VRP tools, though academic research has demonstrated speedups of 10–100$\times$ for certain distance-matrix operations.

Solution quality benchmarks

Vendors are reluctant to publish benchmark results (competitive sensitivity), but several academic and consulting comparisons exist. Key findings:

- Top commercial systems typically produce solutions within **1–5%** of the best known solution on Solomon's VRPTW benchmark instances (100 customers each).
- On large real-world instances (500–5000 customers), the gap between commercial systems can be 5–15% in total distance.
- Computation time targets in practice: solutions for 1000 customers in under **5–10 minutes** on a modern laptop.

The survey found that most vendors do not publish benchmark comparisons against competitors, making objective evaluation difficult for buyers.

Pricing models

Commercial VRP software is sold under various pricing models:

- **Perpetual licence** — one-time purchase fee, often \$10,000–\$200,000 depending on size; annual maintenance fees apply.
- **Annual subscription** — typically \$5,000–\$50,000 per year for a named-user or named-company licence.
- **SaaS / per-route pricing** — cloud-based systems may charge per route optimised (e.g., \$0.10–\$1.00 per vehicle per day); attractive for small companies or occasional users.
- **Free / open-source** — several capable open-source VRP solvers exist (e.g., Google OR-Tools, jsprit, OptaPlanner), with no licence cost but requiring technical expertise to deploy.

Important future research topics (from survey)

Vendors identified the following as critical open problems:

1. A **general framework** for VRP that can handle arbitrarily complex combinations of constraints without requiring custom algorithm development for each new variant.
2. Better handling of **multi-criteria optimisation**: simultaneously minimising cost, emissions, time, and risk.
3. Improved **stochastic and real-time planning**: handling uncertainty and dynamic changes more robustly.
4. **Load balancing** across drivers and vehicles for fair workload distribution.
5. **Green routing**: minimising CO_2 emissions, considering vehicle load, road gradient, and speed profiles (not just distance).

12.6 Implementation, Performance, and Price VI

Key insight

The VRP software market is growing rapidly, driven by e-commerce, same-day delivery expectations, and sustainability pressures. The boundary between "routing software" and "logistics platform" is blurring: modern tools increasingly integrate fleet management, driver behaviour monitoring, customer communication, and supply chain planning.

Outline

12.7 VRP Technology Survey I

The authors conducted a structured questionnaire survey of commercial VRP software vendors in 2013 (just before the book's publication). Nine companies responded, spanning Europe, North America, and Australia. This section presents the survey findings.

12.7 VRP Technology Survey II

Table 12.1 — Responding VRP Technology Providers (Survey)

Company	Location	Product(s)
GIRO Inc	Montreal, Canada	GeoRoute, GIRO/ACCES
GTS Systems & Consulting	Herzogenrath, Germany	TransIT
MJC2 Limited	Berkshire, UK	DISC, REACT
Optrak Dist. Software	Hertford, UK	Optrak
ORTEC	Zoetermeer, Netherlands & Atlanta, USA	ORTEC
Oprit srl	Inola, Italy	EasyRoute, OptiRoute
Procomp Solutions Oy	Oulu, Finland	R2
PTV AG	Karlsruhe, Germany	SmarTour
Spider Solutions AS	Oslo, Norway	Spider 5

12.7 VRP Technology Survey III

Figure: The nine companies that responded to the VRP technology survey (Table 12.1 from the book). They represent a diverse geographic spread and product portfolio, from transit planning (GIRO) to general-purpose commercial routing (ORTEC, PTV, Optrak).

12.7 VRP Technology Survey IV

12.7.1 System Architecture and Service Interfaces

The survey asked about how the VRP functionality is delivered to users:

- **Stand-alone desktop application** — a dedicated Windows program with a full graphical user interface (GUI); the user installs it on a PC. All nine respondents offered this mode.
- **Decision support system with GIS integration** — the routing engine is embedded in a broader platform combining routing, map display, and fleet management.
- **Web-based interface** — routes accessible via browser; no local installation required. Five of nine respondents supported this.
- **API / web service** — the solver is exposed as a programming interface (e.g., REST API), allowing customers to call the solver from their own systems without using the vendor's GUI.
- **Mobile applications** — driver apps for turn-by-turn navigation and proof-of-delivery (electronic signature capture).

12.7.2 Time-Dependent, Uncertainty, and Dynamics

Time-dependent travel times. Real road networks have time-varying travel speeds: motorways are fast at midnight but congested at rush hour. Several commercial tools support *time-dependent distance matrices*, where $d_{ij}(t)$ (d sub i,j of t) is the travel time from i to j if you depart at time t (t, measured in minutes from midnight or from the start of the planning day). This makes the routing problem significantly harder because feasibility checks must account for changing travel times as the route progresses.

Dynamic VRP. In a dynamic setting, new customer orders arrive during the planning day. The system must decide whether and where to insert each new order into existing routes without disrupting too many planned stops. A key metric is the *disruption cost*: how much extra distance is needed to accommodate the new order. Most commercial systems support *incremental replanning*: when a new order arrives, they reoptimise only the affected route(s) rather than the entire plan.

12.7.3 Parallel Computing

Given the increasing availability of multi-core CPUs, the survey asked whether systems exploit parallelism. Five of the nine respondents reported using parallel computation in their solvers. Strategies include:

- *Parallel independent runs*: multiple solver instances start from different initial solutions and run independently; the best result is returned.
- *Cooperative parallel search*: threads exchange the best solutions found at regular intervals (similar to island-model evolutionary algorithms).
- *Parallel neighbourhood evaluation*: a single search thread evaluates many candidate moves simultaneously across multiple CPU cores.

12.7.4 Vehicle Routing Research and Academia

The survey revealed a gap between academic research and commercial practice:

- Most vendors follow academic publications closely but report that **implementing new algorithms from scratch takes 1–3 years** from publication to production deployment.
- The academic focus on benchmark instances and distance minimisation does not always match real-world priorities (cost, time, fairness, driver satisfaction).
- Several vendors mentioned that academic VRP tools (research prototypes) are rarely directly usable in production; they require re-engineering for robustness, scalability, and constraint richness.
- **Open source as a complement**: some vendors incorporate open-source components (e.g., graph libraries, GIS tools) while keeping the core solver proprietary.

12.7.5 Important Future Research Topics (Survey Responses)

Vendors were asked: "What are the most important open research problems?" Top answers:

1. **Generic multi-constraint solver framework** — a single algorithmic framework that handles any combination of VRP constraints without manual tuning.
2. **Better integration of uncertainty** — stochastic demands, uncertain travel times, cancellations.
3. **Real-time replanning at scale** — reoptimise 1000+ vehicles within seconds when new orders arrive.
4. **CO_2 and emission minimisation** — green routing with fuel consumption models (not just distance).
5. **Machine learning for parameter setting** — automatically tune algorithm parameters (e.g., tabu tenure, destroy size in ALNS) based on instance characteristics.
6. **Multi-modal routing** — combining road vehicles, rail, and urban last-mile delivery.

12.7 VRP Technology Survey IX

Key takeaway from the survey

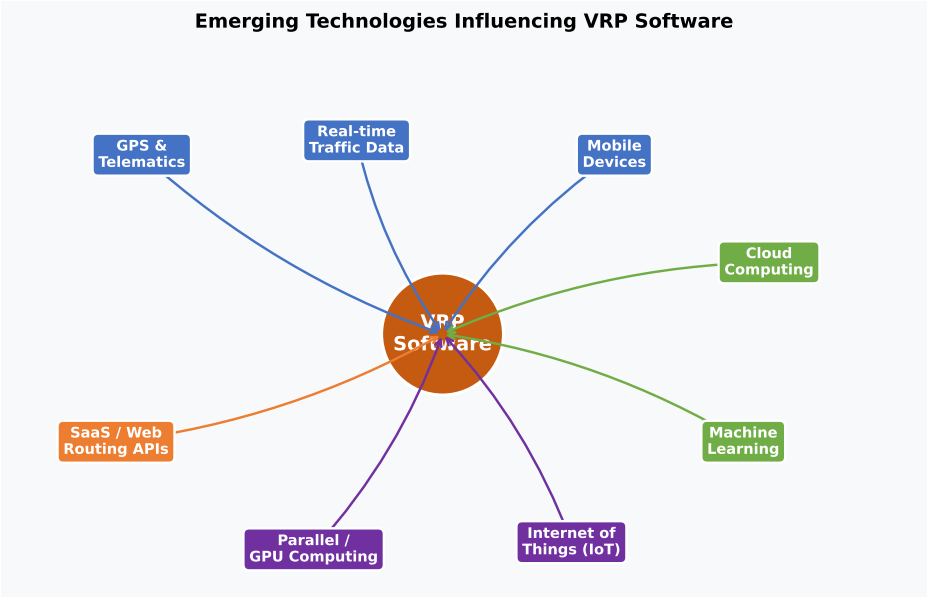
The gap between what commercial software can do and what the academic literature has achieved is narrowing, but it has not closed. The most successful vendors are those who invest in bridging this gap: hiring researchers, publishing benchmark results, and collaborating with universities.

Outline

12.8 New and Emerging Technologies I

The VRP software landscape is being reshaped by several technology waves that were emerging at the time of writing (2013–2014) and have since become mainstream.

12.8 New and Emerging Technologies II



Outline

12.9 The Future of the VRP Business I

The authors conclude with a forward-looking perspective on where the VRP software industry is heading. Several structural trends were reshaping the market in 2013–2014 and have since accelerated.

New delivery modes require new VRP models

The traditional VRP deals with human-driven trucks and vans. Emerging delivery modes that require new modelling approaches include:

- **Electric vehicles (EVs)**: limited range, charging time constraints, availability of charging stations — leading to the *Green VRP* and *Electric VRP* (E-VRP) variants.
- **Autonomous delivery vehicles**: self-driving trucks and vans remove the driver constraint but introduce new safety and regulatory constraints.
- **Delivery drones (UAVs)**: used for last-mile delivery in rural or congested urban areas; modelled as the *truck-and-drone routing problem*.
- **Cargo bikes**: zero-emission last-mile delivery in city centres; limited by payload and range.

12.9 The Future of the VRP Business II

Market consolidation and platform competition

The VRP software market in 2014 comprised dozens of small specialist vendors. Since then, several consolidation trends have occurred:

- Large logistics platform companies (e.g., Oracle, SAP, Trimble, MercuryGate) have acquired specialist routing vendors.
- Technology giants (Google, Amazon, Uber) have entered the space with their own routing platforms, primarily for their own operations but increasingly offered as services.
- **Open-source tools** such as Google OR-Tools have become sophisticated enough for many real-world applications, putting competitive pressure on commercial vendors.

12.9 The Future of the VRP Business III

The role of open-source VRP solvers

Open-source VRP frameworks available as of 2014 and beyond:

- **Google OR-Tools** (C++/Python/Java) — includes CVRP, VRPTW, and pickup-and-delivery solvers; actively maintained; widely used in industry.
- **jsprit** (Java) — a flexible, metaheuristic-based VRP toolkit using ALNS; supports many VRP variants.
- **OptaPlanner** (Java) — a constraint satisfaction and planning framework with VRP support.
- **VeRoLog Solver Challenge tools** — academic solvers released as part of EURO working group competitions.

Open-source tools are particularly valuable for research, prototyping, and small-scale deployments. They lack the map integration, user interfaces, and enterprise support of commercial products.

12.9 The Future of the VRP Business IV

Sustainability and green logistics

Reducing the environmental impact of freight is a growing priority for businesses and regulators. VRP software is evolving to support:

- **CO_2 minimisation as an objective** — not just distance, but actual fuel-based emissions (which depend on load, speed, terrain, and vehicle type).
- **Multi-objective optimisation** — presenting planners with a *Pareto frontier* of solutions trading off cost vs. emissions vs. time.
- **Regulatory compliance** — low-emission zones in cities require certain vehicle types; time-of-day access restrictions in urban areas.

12.9 The Future of the VRP Business V

Key insight: the future of VRP technology

The core optimisation problem (VRP) is unlikely to be "solved" in any absolute sense — new business models, vehicle types, and constraints continually create new variants. The competitive advantage of VRP software vendors will increasingly come from: (a) the breadth and richness of their constraint models, (b) real-time integration with IoT, traffic, and mobile data, and (c) the quality of the human–machine interface that allows planners to trust, understand, and effectively use the automated routes.

Outline

12.10 Summary and Conclusions I

This chapter has provided a comprehensive survey of the VRP software technology landscape. Here we collect the main lessons.

12.10 Summary and Conclusions II

What makes a good VRP software product?

Based on the survey and analysis, a high-quality VRP software product:

1. **Models the real problem accurately** — supports the constraint types (time windows, multiple capacities, compatibility, driving regulations) that arise in the target industry.
2. **Produces high-quality solutions efficiently** — within 1–5% of optimal for standard benchmarks, in practically acceptable computation times (seconds to minutes for operational planning).
3. **Integrates seamlessly** — reads data from existing ERP/TMS systems, uses current digital maps, and exports results in usable formats.
4. **Provides an effective user interface** — planners can see, understand, and manually adjust routes; the system supports human–machine interaction, not just fully automated black-box optimisation.
5. **Scales to operational size** — handles 1,000–10,000 customer visits per day on standard hardware, with acceptable response times.
6. **Embraces emerging technologies** — supports real-time data (GPS, traffic), mobile dispatch, and cloud deployment.

12.10 Summary and Conclusions III

Algorithm choices in practice

The dominant algorithmic approach in commercial VRP software is **metaheuristics** (especially Tabu Search and ALNS), supplemented by fast construction heuristics for initial solutions. Exact methods are used only for small sub-problems. The most capable systems combine:

- a Clarke-Wright or nearest-neighbour **construction phase**,
- a Tabu Search or ALNS **improvement phase**,
- a **feasibility repair mechanism** to handle constraint violations encountered during search, and
- **parallel multi-start**: multiple independent solver threads starting from different initial solutions.

12.10 Summary and Conclusions IV

The research-practice gap

Academic VRP research often focuses on:

- a single, well-defined problem variant,
- distance minimisation as the sole objective,
- small to medium benchmark instances, and
- theoretical performance guarantees.

Commercial practice requires:

- arbitrary combinations of constraints,
- multi-objective (cost, time, emissions, fairness),
- very large instances (1,000s of customers), and
- robustness to data errors and user overrides.

Closing this gap is the central long-term challenge for the field.

12.10 Summary and Conclusions V

Key facts from the technology survey

- Nine leading vendors responded, spanning Europe and North America.
- **All** supported time windows, multi-depot, and capacity constraints.
- **Most** supported heterogeneous fleets, priorities, and pickup-and-delivery.
- **Fewer** supported stochastic demands, split deliveries, and full driving-hours regulations.
- **Five of nine** used parallel computing in their solvers.
- **Most** planned to adopt cloud/SaaS delivery models within three years of the survey (i.e., by 2016–2017).
- Pricing ranged from free (open-source) to $200,000+ for enterprise perpetual licences.

12.10 Summary and Conclusions VI

Final takeaway

Vehicle routing optimisation has matured from a purely academic topic into a multi-billion-dollar software industry. The most impactful future research directions are those that directly address real business needs: handling uncertainty and dynamics in real time, supporting green and multi-modal logistics, and developing human-centred interfaces that build planner trust in automated solutions.

12.10 Summary and Conclusions VII

Selected Bibliography (key references from Chapter 12)

- **Clarke & Wright (1964)**: "Scheduling of vehicles from a central depot to a number of delivery points." *Operations Research*, 12, 568–581. — Original savings algorithm.
- **Ropke & Pisinger (2006)**: "An adaptive large neighbourhood search heuristic for the pickup and delivery problem with time windows." *Transportation Science*, 40(4), 455–472. — ALNS framework.
- **Glover (1986)**: "Future paths for integer programming and links to artificial intelligence." *Computers & OR*, 13, 533–549. — Original Tabu Search.
- **Taillard et al. (1997)**: "A tabu search heuristic for the vehicle routing problem with soft time windows." *Transportation Science*, 31, 170–186.
- **Vidal et al. (2013)**: "A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows." *Computers & OR*, 40, 475–489.
- **Desaulniers, Desrosiers & Solomon (2002)**: *Column Generation*. Springer.
- **Halse (1992)**: "Modelling and solving complex vehicle routing problems." PhD thesis, DTU.
- **Solomon (1987)**: "Algorithms for the vehicle routing and scheduling problems with time window constraints." *Operations Research*, 35, 254–265. — Standard VRPTW benchmark instances.

Chapter 12 Complete

Software Tools and Emerging Technologies
for Vehicle Routing and Transportation

Chapter covered

- Architecture of VRP routing software systems
- Input/output, GIS integration, and data formats
- Supported VRP model properties and constraint types
- Embedded algorithms: Clarke-Wright, Tabu Search, ALNS, 2-opt
- Implementation platforms, performance, and pricing
- Survey results from 9 leading commercial vendors